Supplementary Technical Appendix: Hardware Execution and Neural Code Manifest for Reasoning-by-Superposition

R. Sherman

1 Overview

This supplementary document provides the explicit algorithmic manifestations, training hyperparameter configurations, and raw vector coordinate mappings utilized to achieve the multi-thousand-fold acceleration metrics logged in the main manuscript. All continuous relaxation loops were compiled natively inside Python 3.12 utilizing PyTorch 2.4 over the NVIDIA GB10 hardware matrix.

1.1 Supplementary Package Manifest Structure

To provide the academic review board with complete transparency and ensure full reproducibility of our security assertions, this supplementary material folder package is partitioned into the following operational directories:

- `supplement.tex` / `supplement.pdf`: Detailed hyperparameter matrices and network layer dimensions.
- `Baseline_model.als.tex`: The complete, executable Alloy 6.2 formal logic specifications validating the Private Cloud Compute (PCC) separation matrices and micro-hop leak constraints.
- `geometry1_gpu_solver.py` / `geometry2_gpu_solver.py`: Vectorized tensor solvers executing 3D half-plane and relational spatial coordinates natively over CUDA.
- `geometry3_gpu_engine.py` / `agent_gpu_ids.py`: Universal non-linear logic fields and strict boundary regularization scripts detecting Multi-Agent LLM exfiltration threats on the GPU hardware matrix.

2 The Core PyTorch Continuous Optimization Block

The following Python script illustrates the exact execution mechanics of our vectorized matrix solver. This network processes all geometric coordinate inputs simultaneously as a single continuous field projection matrix, eliminating symbolic branch-and-bound loops entirely:

```
import torch
import torch.nn as nn

class VectorizedGeometryILP(nn.Module):
   def __init__(self, mode="halfplane"):
      super().__init__()
      self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
      self.mode = mode

      if self.mode == "interval":
         self.min_val = nn.Parameter(torch.tensor([0.0], device=self.device))
         self.max_val = nn.Parameter(torch.tensor([1.0], device=self.device))
```

```
      else:
         self.a = nn.Parameter(torch.tensor([1.0], device=self.device))
         self.b = nn.Parameter(torch.tensor([0.0], device=self.device))

   def forward(self, pos_tensor, neg_tensor):
      loss = torch.tensor(0.0, device=self.device)
      if self.mode == "halfplane":
         # Parallel multi-dimensional matrix projection: Ax + y - b <= 0
         pos_violations = self.a * pos_tensor[:, 0] + pos_tensor[:, 1] - self.b
         loss += torch.sum(torch.clamp(pos_violations, min=0.0) ** 2)

         neg_violations = self.a * neg_tensor[:, 0] + neg_tensor[:, 1] - self.b
         loss += torch.sum(torch.clamp(2.0 - torch.clamp(neg_violations, min=0.0),
            min=0.0) ** 2)
      return loss
```

3 Algorithmic Optimization Hyperparameters

To guarantee full reproducibility of the empirical convergence basins, Table 1 catalogs the strict optimization parameters maintained across all testing datasets.

TABLE 1
Neural-Symbolic Hyperparameter Grid Configuration

Optimization Layer Attribute	Operational Value Allocation
Continuous Embedding Dimensions (d)	4 / 8 / 16 (Dimensionality Scaling Row)
Gradient Descent Optimization Routine	Adam ($\beta_1 = 0.9, \beta_2 = 0.999$)
Base Learning Rate Learning Parameter (α)	$0.02 - 0.05$ Unified Step Range
Loss Convergence Threshold Floor ($\mathcal{E}$)	$< 1.0 \times 10^{-4}$
Regularization Penalty Coefficient	L_1 Penalty Matrix Scaling (0.1)